2ceeaf51-dc03-4444-b425-719f0ff1fbb5

<prev_user>

<request>

You are working on an issue in the 'Azure/azure-sdk-for-js' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/azure-sdk-for-js/azure-sdk-for-js (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-16T16:54:42.177Z</current_datetime>

Consider the following problem statement:

<problem_statement>
address @Azure/azure-sdk-for-js/issues/36230

Revisit all promises in the event hubs library and make sure not a single one of them is dangling. Use modern syntax and maintain current behavior
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

Excellent! Now let me make a final summary of the work done:

Summary

I have successfully addressed issue #36230 by identifying and fixing all dangling promises in the @azure/event-hubs library. Here's what was accomplished:

Changes Made:

1. **Fixed Event Handlers with Dangling Promises** (8 handlers across 3 files):
   - `connectionContext.ts`: Fixed `onDisconnected` handler by wrapping async operations in IIFE with error handling; removed unnecessary `async` from `protocolError` and `error` handlers
   - `eventHubSender.ts`: Fixed `_onAmqpClose` and `_onSessionClose` handlers by re: "occured" → "occurred"

4. **Fixed Timer Loop**:
   - Added final `.catch()` in `util/timerLoop.ts` to handle promise returned from `.finally()`

Key Achievements:

✅ **Minimal Changes**: Only 8 files changed with 133 insertions and 115 deletions
✅ **No Behavior Changes**: All fire-and-forget operations remain fire-and-forget
✅ **Proper Error Handling**: All async operations in event handlers now have error handling
✅ **Linting Clean**: No new linting errors introduced (0 errors, 46 pre-existing warnings remain)
✅ **Type Safety**: All event handlers properly match the `OnAmqpEvent` type signature
✅ **Modern Syntax**: Used async/await patterns consistently

Root Cause Addressed:

The main issue was that event handlers were declared as `async` functions but registered with rhea-promise which expects `OnAmqpEvent: (context: EventContext) => void`. When async functions were used as event handlers, they returned promises that were never awaited, creating dangling promises that could reject and crash the application.

The fixes ensure all promises are either:

- Properly awaited in appropriate contexts
- Wrapped with `.catch()` for fire-and-forget operations
- Enclosed in IIFE with outer catch for complex async event handlers

This prevents UnhandledPromiseRejection errors like the one reported in #36230 where `subscription.close()` timeouts would crash AWS Lambda functions.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Azure/azure-sdk-for-js' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/azure-sdk-for-js/azure-sdk-for-js (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-16T18:23:26.927Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix dangling promises in Event Hubs library to prevent UnhandledPromiseRejection errors
</pr_title>

<pr_description>
Problem

The Event Hubs library had several locations where promises could become unhandled, leading to `UnhandledPromiseRejection`

````
errors that crash applications. This was reported in #36230
where `subscription.close()` timeout errors would cause
unhandled promise rejections that terminated AWS Lambda
functions.

The root cause was that event handlers were declared as `async`
functions but registered with rhea-promise which expects
`OnAmqpEvent` type: `(context: EventContext) => void`. When
async functions are used as event handlers, they return promises
that are never awaited, creating dangling promises.

## Changes

### 1. Fixed Event Handlers with Dangling Promises

**connectionContext.ts**
- Wrapped the complex async `onDisconnected` handler in an IIFE
(Immediately Invoked Function Expression) with outer `.catch()`
to handle any unhandled errors
- Removed unnecessary `async` keyword from `protocolError` and
`error` handlers that only perform synchronous logging

**eventHubSender.ts**
- Removed `async` keyword and `await` from `_onAmqpClose` and
`_onSessionClose` handlers, making them synchronous functions
that start fire-and-forget async operations with proper error
handling

**partitionReceiver.ts**
- Changed `onClose` and `onSessionClose` functions to properly
`await` the close operations instead of returning the promise
- Wrapped handler invocations with `.catch()` to handle any
errors from the async operations

### 2. Modernized Promise Patterns

Converted `getPartitionIds()` methods in both
`eventHubProducerClient.ts` and `eventHubConsumerClient.ts` from
legacy `.then()` chains to modern `async/await` syntax:

```typescript
// Before
getPartitionIds(options = {}): Promise<Array<string>> {
  return
this._context.managementSession!.getEventHubProperties(...)
    .then((eventHubProperties) =>
eventHubProperties.partitionIds);
````
````

```
}

// After
async getPartitionIds(options = {}): Promise<Array<string>> {
  const eventHubProperties = await
this._context.managementSession!.getEventHubProperties(...);
  return eventHubProperties.partitionIds;
}
```

3. Enhanced Error Handling in Background Operations

- Added clarifying comments for fire-and-forget background loops in `eventHubBufferedProducerClient.ts` and `batchingPartitionChannel.ts`
- Improved error messages to be more specific about what failed
- Fixed typo: "occured" → "occurred"
- Added final `.catch()` in `util/timerLoop.ts` to handle promise returned from `.finally()`

Impact

✅ **No Behavior Changes**: All fire-and-forget operations remain fire-and-forget. Changes only add error boundaries to prevent crashes.

✅ **Better Error Handling**: Event handler errors are now properly caught and logged instead of becoming unhandled promise rejections.

✅ **Type Safety**: All event handlers now properly match the `OnAmqpEvent` type signature.

✅ **Minimal Changes**: Only 8 files changed with 133 insertions and 115 deletions.

✅ **Linting Clean**: All linting passes with no new errors introduced.

Testing

- Existing tests should pass without modification as no behavior was changed
- Linting completes successfully with no new errors (46 pre-existing warnings remain unchanged)
- Code formatted with prettier

Fixes #36230

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_reviews>

<comment_thread>
<file>sdk/eventhub/event-hubs/src/util/timerLoop.ts:41-45</file>
<original_commit>460526d</original_commit>

<comment_new>
<comment_id>2436976115</comment_id>
<author>@jeremymeng</author>
do we want to log the errors?
</comment_new>

<comment_new>
<comment_id>2436976115</comment_id>
<author>@deyaaeldeen</author>
@copilot make sure to log any caught errors
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:

```
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```